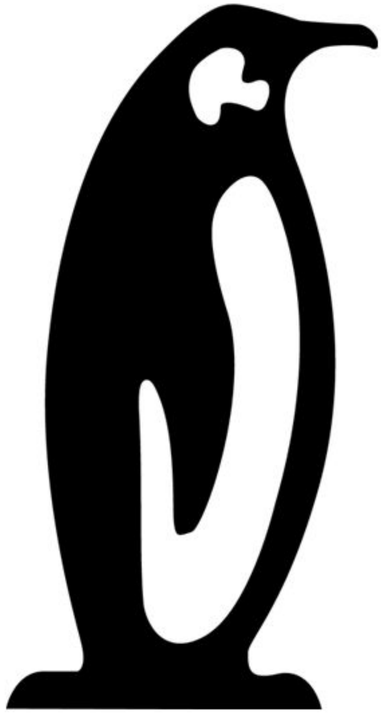


Linux Plumbers Conference

Vienna, Austria | September 18-20, 2024

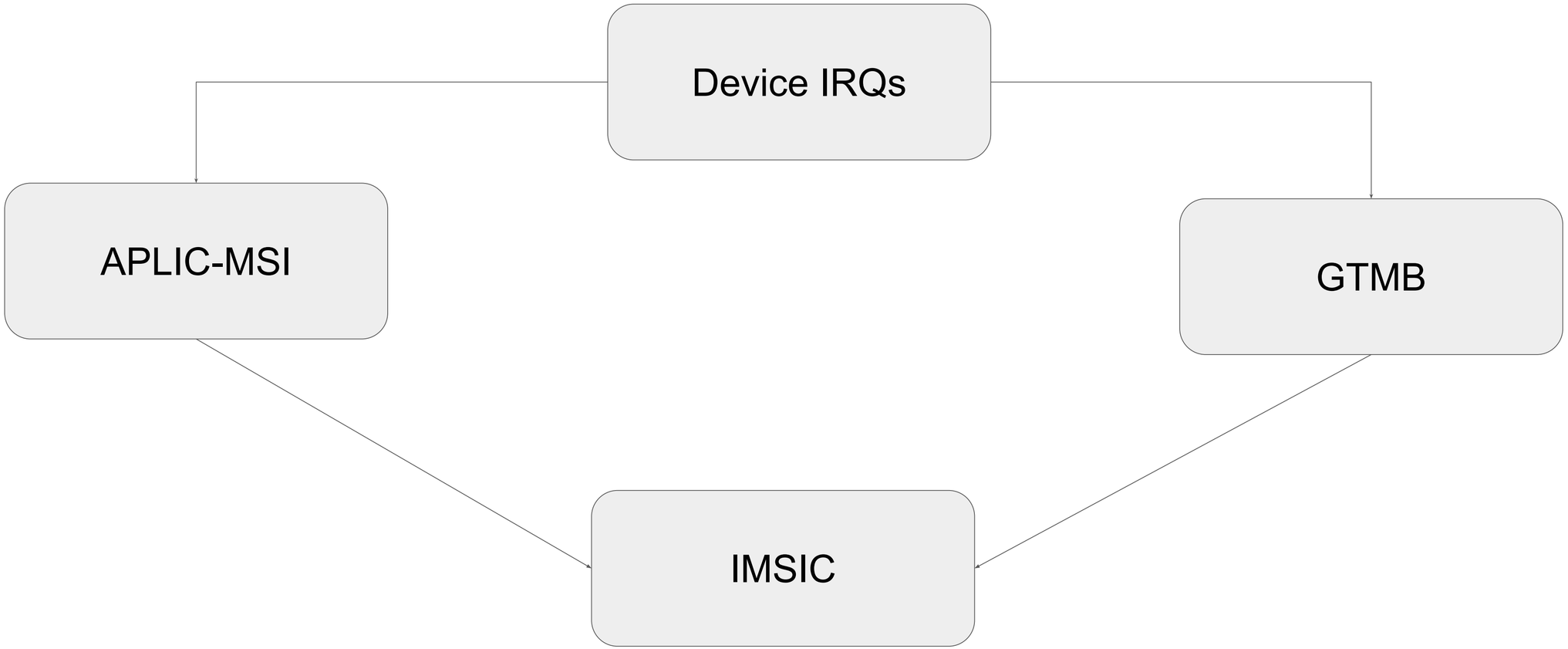
MSI support in ACPI without APLIC

Atish Patra, Rivos

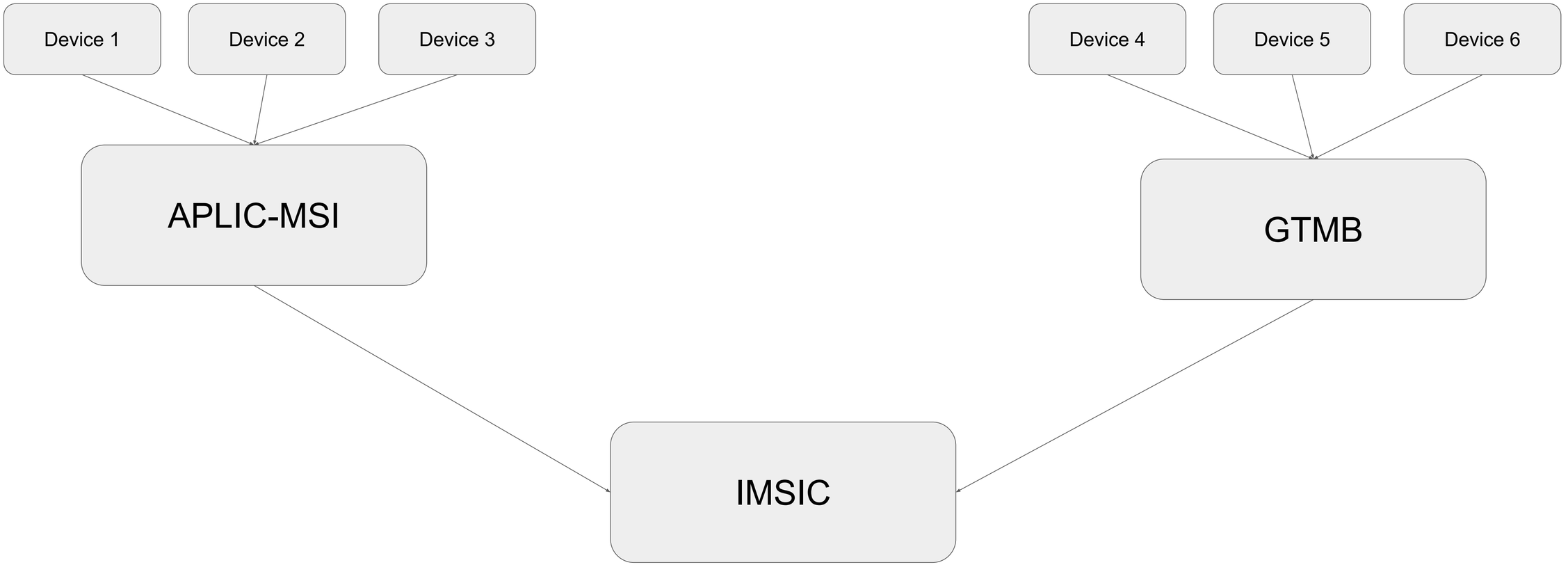


LINUX PLUMBERS CONFERENCE | Vienna, Austria
Sept. 18-20, 2024

Current IRQ domain hierarchy in RISC-V



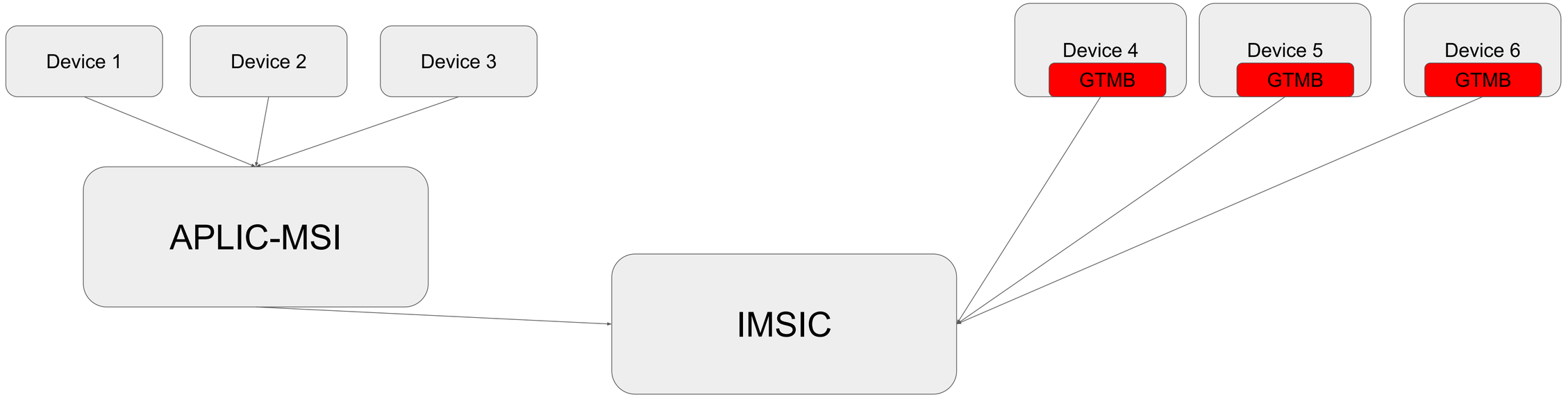
Current GTMB design



Concerns from MarcZ (Relayed via Bjorn/Matt)

- Abstraction of individual device with a single driver
  - It would be good move the abstraction closer to the device
- There are two entities (ASL and Linux kernel) access the device registers. There may be possible race condition.
  - Race condition scenario
    - Linux gets the interrupt and Interrupt handler gets invoked
    - Work is delegated to Bottom half which is trying to access these registers
    - User changes the affinity and MSI target address/data changed.
    - MSI config callbacks gets invoked - GTMB drivers gets invoked
    - GTMB driver modifies the MSI config. This will affect the next interrupt.
    - This should be no different than APLIC (or any other IC) modifying the MSI configs. (edited)
    - The HW should be able to arbitrate that. Isn't it ?

Proposed GTMB design



1. Every device object will have a GTMB/_GMA entry for its own MSI configuration register
2. Device driver will register itself as device MSI domain or request_irq based on _GMA is present or not
3. The device is no longer associated with a GSI and acts as device MSI domain.

Pros: The MSI configuration belongs to the device driver. The device can define some other register for some custom special handling purpose.

Cons: Every existing driver for a device that wants MSI capability needs to be modified. Need more investigation for PoC.
Reality - GED driver must be changed. There is no common GPIO chip driver. So we need to develop one for Rivos anyways.
Any other future platform device (may be due to errata in PCIe) - can also use this easily for discovering MSI configs.

Global System Interrupts (GSI)

- Global System Interrupts (GSIs) are ACPI representations for interrupts in ACPI namespace
- A platform with MSI requires a GSI(IRQ) to MSI bridge for ACPI based drivers using GSI
  - I/O APIC (x86), APLIC (RISC-V), GIC (ARM64) act as the bridge
- In RISC-V, APLIC is an optional standard hardware specification
- A platform can also be built without an IRQ-to-MSI bridge if all devices are MSI capable
  - Interrupts for PCIe based devices will be managed by PCIe controller
  - What about platform devices capable of MSI that need to use ACPI GSI mechanism?

What’s missing ?

- Currently, only wired interrupts can be represented via GSI in ACPI
- Platform MSI device is capable of triggering MSIs directly
- OSPM needs a mechanism to update the platform MSI device about Interrupt vector and target hart
- This can change during irq affinity change/ irq mask/unmask operation
- Solution:
  - Develop/Maintain your custom driver
  - Reuse already existing ACPI drivers
    - However GSIs can not be mapped to MSIs without a dedicated IRQ-to-MSI bridge hardware.

GSI-to-MSI Bridge(GTMB) Proposal

- An ACPI device that provides abstraction for GSI to MSI conversion functionality at each device
- DSDT table defines the register layout per GSI
- The register layout is flexible and scalable as per platform designer
- There is no MADT defined as it can not scale to any number of GSIs
  - Early probing of GTMB is not supported as there is no MADT
- Must be a child irq domain of architecture specific MSI controller (i.e. IMSIC)
- Draft proposal
  - https://docs.google.com/document/d/1If9PDwLgAwEExzILDf3JWstCrt61gdV6k3L33iUVbPk/edit

GTMB layout

GSI-0x61

```
Device (GTMB) {              // GTMB Device object
    Name (_HID, "ACPI0019")  // _HID: Hardware ID
    Name (_UID, Zero)        // _UID: Unique
    Name (_GSB, 0x61)  // _GSB: Global System Interrupt Base
    Name (_NGI, 0x04)  // _NGI: Number of Global System Interrupts
supported
    Name (_GMA, Package (0x02) {
            0x00, //_GMA version number
      Package() {
            0x61, //GSI Number
            ResourceTemplate () { //MSI ADDR LOW
                Register(SystemMemory, 32, 0, 0x10220000, DWORD)
            },
            ResourceTemplate () { //MSI ADDR HIGH
                Register(SystemMemory, 32, 0, 0x10220004, DWORD)
            },
            ResourceTemplate () { //MSI DATA
                Register(SystemMemory, 32, 0, 0x10220008, DWORD)
            },
            ResourceTemplate () { //MSI Enable
                Register(SystemMemory, 1, 0, 0x1022000C, DWORD)
            },
            ResourceTemplate () { //MSI RetriggerValue
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { //MSI Retrigger
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { //MsiTriggerTypeEdgeRising
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { //MsiTriggerTypeEdgeFalling
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { //MsiTriggerTypeLevelHigh
                Register(SystemMemory, 0, 0, 0, 0)
            },
```

GSI-0x62

```
Package() {
            0x62, //GSI Number
            ResourceTemplate () { //MSI ADDR LOW
                Register(SystemMemory, 32, 0, 0x10420000, DWORD)
            },
            ResourceTemplate () { //MSI ADDR HIGH
                Register(SystemMemory, 32, 0, 0x10420004, DWORD)
            },
            ResourceTemplate () { //MSI DATA
                Register(SystemMemory, 32, 0, 0x10420008, DWORD)
            },
            ResourceTemplate () { //MSI Enable
                Register(SystemMemory, 1, 0, 0x1042000C, DWORD)
            },
            ResourceTemplate () { //MSI RetriggerValue
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { //MSI Retrigger
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { // MsiTriggerTypeEdgeRising
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { // MsiTriggerTypeEdgeFalling
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { // MsiTriggerTypeLevelHigh
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { // MsiTriggerTypeLevelLow
                Register(SystemMemory, 0, 0, 0, 0)
            },
            ResourceTemplate () { // MsiTriggerType
                Register(SystemMemory, 0, 0, 0, 0)
            },
        }

    )
}
```

Current status of the proposal

- PoC is complete based on the current draft
  - Based on Qemu virt device with GED device
- Discussed/Reviewed internally within RISC-V Platform Runtime Services (PRS) TG
  - Made it generic instead of RISC-V specific (based on the feedback received within PRS)
- Presented within ASWG working committee
  - Ongoing discussions

Demo - I

```
root@debian:~# cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7
 10:       1519       2245       3756       2061       2512       1731       1702       1954  RISC-V INTC   5 Edge      riscv-timer
 11:        314          0          0          0          0          0          0          0  APLIC-MSI-RSCV0002:00  10 Level   -fasteoi   ttyS0
 19:         78          0          0          0          0          0          0          0  APLIC-MSI-RSCV0002:00   8 Level   -fasteoi   virtio0
 20:          0          0          0          0          0          0          0          0  GTMB-MSI-ACPI0019:00   0 Edge    -fasteoi   ACPI:Ged
 25:          0          0          0          0          0          0          0          0  PCI-MSIX-0000:00:03.0   0 Edge      virtio3-config
 26:        118          0          0          0          0          0          0          0  PCI-MSIX-0000:00:03.0   1 Edge      virtio3-req.0
 27:          0         41          0          0          0          0          0          0  PCI-MSIX-0000:00:03.0   2 Edge      virtio3-req.1
 28:          0          0        345          0          0          0          0          0  PCI-MSIX-0000:00:03.0   3 Edge      virtio3-req.2
 29:          0          0          0         94          0          0          0          0  PCI-MSIX-0000:00:03.0   4 Edge      virtio3-req.3
 30:          0          0          0          0        274          0          0          0  PCI-MSIX-0000:00:03.0   5 Edge      virtio3-req.4
 31:          0          0          0          0          0        123          0          0  PCI-MSIX-0000:00:03.0   6 Edge      virtio3-req.5
 32:          0          0          0          0          0          0         69          0  PCI-MSIX-0000:00:03.0   7 Edge      virtio3-req.6
 33:          0          0          0          0          0          0          0        182  PCI-MSIX-0000:00:03.0   8 Edge      virtio3-req.7
 34:          0          0          0          0          0          0          0          0  PCI-MSIX-0000:00:01.0   0 Edge      virtio1-config
 35:          0          0          0          0          0          0          0          0  PCI-MSIX-0000:00:01.0   1 Edge      virtio1-requests
 36:          0          0          0          0          0          0          0          0  PCI-MSIX-0000:00:02.0   0 Edge      virtio2-config
 37:          0          0          0          0          0          0          0          0  PCI-MSIX-0000:00:02.0   1 Edge      virtio2-requests
IPI0:        98         47         69         43         84         71         94         70  Rescheduling interrupts
IPI1:      3522       3548       1961       2297       2277       2362       3491       3651  Function call interrupts
IPI2:         0          0          0          0          0          0          0          0  CPU stop interrupts
IPI3:         0          0          0          0          0          0          0          0  CPU stop (for crash dump) interrupts
IPI4:         0          0          0          0          0          0          0          0  IRQ work interrupts
IPI5:         0          0          0          0          0          0          0          0  Timer broadcast interrupts
```

Demo - II

```
root@debian:~# cat /sys/kernel/debug/irq/irqs/20
handler:  handle_fasteoi_irq
device:   ACPI0019:00
status:   0x00000401
            _IRQ_NOPROBE
istate:   0x00004020
            IRQS_ONESHOT
ddepth:   0
wdepth:   0
dstate:   0x02400201
            IRQ_TYPE_EDGE_RISING
            IRQD_ACTIVATED
            IRQD_IRQ_STARTED
            IRQD_DEFAULT_TRIGGER_SET
node:     0
affinity: 0-7
effectiv: 1
domain:  \_SB.GMB0-16
 hwirq:   0x0
 chip:    GTMB-MSI-ACPI0019:00
  flags:   0x15
             IRQCHIP_SET_TYPE_MASKED
             IRQCHIP_MASK_ON_SUSPEND
             IRQCHIP_SKIP_SET_WAKE
 parent:
    domain:  imsic-5
     hwirq:   0x14
     chip:    IMSIC
      flags:   0x14
                 IRQCHIP_MASK_ON_SUSPEND
                 IRQCHIP_SKIP_SET_WAKE
     target_cpu      :     1
     target_local_id :     3
     is_reserved     :     0
     is_enabled      :     1
     is_move_pending :     0
root@debian:~#
```



Linux Plumbers Conference

Vienna, Austria | September 18-20, 2024